

Bloquant

Perm.Gather

true : se déclare prêt PUIS attend « échange conclu » ; false : retire son prêt. validate(t) = ready(true, t).

`trade:accept() → bool``trade:acceptFrom([nom] [, timeoutSec=10]) → bool`

Perm.Gather

accept() accepte la demande reçue (non bloquant) ; acceptFrom attend une demande (de nom, ou de n'importe qui), l'accepte et attend l'ouverture.

`trade:pending() → table|nil`

Demande d'échange reçue en attente : { id, name, type, age } ou nil.

`trade:cancel() → bool``trade:refuse() → bool``trade:isOpen() → bool`

Annule l'échange / refuse la demande reçue ; isOpen() = un échange est-il ouvert ?

`trade:autoAcceptTeam([on=true [, autoReady=true]]) → bool`

Perm.Multi

Mode MULE : accepte automatiquement les échanges des bots de ton équipe (et se met prêt), le temps du script.

```
-- Chef : donner 100 Blé à la mule présente sur la carte
if trade:withPlayer("Mule-Bob") then
    trade:putItem(289, 100)
    trade:ready()           -- prêt + attente de la conclusion
end

-- Mule (dans son propre script) : tout accepter de l'équipe
trade:autoAcceptTeam(true)
```

Copier

Module merchant: — marchands (compat SnowBot, neutre)

Module inactif chez SnowBot aussi : présent pour qu'un script qui l'appelle se charge.
Avertissement (une fois) et sentinelles SnowBot.

```
merchant:open(nom) → nil  
merchant:buyItem(gid, quantité) → false  
merchant:priceItem(gid) → -1  
merchant:quantityItem(gid) → -1  
merchant:existItem(gid) → false
```

Aucun effet : open → nil, buyItem / existItem → false, priceItem / quantityItem → -1.

Modules zaap: & zaapi: — téléport

Téléports par le zaap / zaapi présent sur la carte courante. Pour un voyage générique, utilise travelTo (qui choisit le zaap tout seul si utile). Noms SnowBot : map:toZaap(mapId [, 1 = zaapi]), map:knownZaap(), map:getNearestZaap(...) (module map:). ⚠ goto est un mot-clé Lua : zaap:goto(x) est une erreur de syntaxe, utilise zaap:go(x).

```
zaap:go(mapId) → bool
```

Bloquant

Marche jusqu'au zaap de la carte courante, l'ouvre et se téléporte vers mapId.

```
zaap:go(212861955)
```

Copier

```
zaap:travel(mapId) → bool
```

Envoie le téléport vers mapId si un dialogue de zaap est DÉJÀ ouvert et propose cette destination (false sinon ou si l'envoi échoue). Non bloquant : pour marcher jusqu'au zaap et attendre l'arrivée, utilise zaap:go.

```
zaap:canGoTo(mapId) → bool
```

true si mapId est une destination du dialogue de zaap ouvert.

`zaap:known()` → `table[]`

Zaaps connus : { mapId } par entrée (vide tant qu'aucun dialogue de zaap n'a été vu depuis la connexion).

`zaapi:go(mapId)` → `bool`

`zaapi:canGoTo(mapId)` → `bool`

Bloquant

Zaapi (Bonta / Brâkmar) de la carte courante : se téléporte vers mapId / destination proposée ?

Métiers & craft (job:, craftItem)

Niveaux de métier et craft complet en un appel. Pour un craft pas à pas façon SnowBot (établi déjà ouvert), voir le module craft:.

`job:level(jobId)` → `number`

Niveau du métier jobId (0 si inconnu / non appris).

`job:all()` → `table`

Table complète { jobId = niveau } de tous les métiers connus.

```
for id, lvl in pairs(job:all()) do print(job:name(id), lvl) end
```

Copier

`job:name(jobId)` → `string`

Nom du métier jobId.

`craftItem(elementId, skillUid, recipe, count)` → `bool`

Bloquant

Perm.Gather

Craft complet en un appel : ouvre l'atelier (elementId et skills[i].uid via map:getInteractives()), pose les ingrédients recipe = { {gid=..., quantity=...}, ... } (qty accepté), règle le nombre count, valide, attend le résultat et ferme.

```
craftItem(ATELIER_ID, SKILL_UID, {
    { gid = INGREDIENT_1, quantity = 4 },
    { gid = INGREDIENT_2, quantity = 1 },
}, 10)
```

[Copier](#)

Module craft: — craft pas à pas (compat SnowBot)

Sur un établi DÉJÀ ouvert (map:useCell / map:useById sur l'atelier, ou goUseInteractive) : poser les ingrédients, régler la quantité, valider. L'établi reste ouvert après ready : ferme-le avec leaveDialog(). Ingrédients pris dans les piles du sac. Permission « Récolte auto » ; false + avertissement si aucun établi n'est ouvert.

```
craft:PutItem(gid, qty) → bool
```

```
craft:putItem(gid, qty) → bool
```

Bloquant

Perm.Gather

Pose qty de l'ingrédient gid ($qty \leq 0$ = tout ce qu'a le sac) et attend l'accusé (≤ 3 s par pile, $\sim 0,3$ s de cadence) ; s'arrête si l'établi se ferme entre deux piles. Une erreur d'échange renvoyée par le serveur n'est PAS comptée comme une pose : la pile refusée ne compte pas et la pose s'arrête là ; true seulement si au moins une pile a vraiment été posée (sinon false). exchange:putItem y est aussi routé quand un établi est ouvert (mêmes règles).

```
craft:ChangeQuantityToCraft(n) → bool
```

```
craft:changeQuantityToCraft(n) → bool
```

Bloquant

Perm.Gather

Nombre de crafts à enchaîner (≥ 1) ; attend la confirmation (≤ 3 s).

```
craft:Ready() → bool
```

```
craft:ready() → bool
```

Bloquant

Perm.Gather

Valide et attend le résultat (≤ 12 s) : true si un objet a été créé (résultat vide = recette invalide). L'établi reste ouvert.

```
craft:isOpen() → bool
```

```
craft:craftItem(elementId, skillId, recipe, count) → bool
```

Un établi est-il ouvert ? / craftItem = le global craftItem (craft complet en un appel).

```
-- Atelier ouvert par le script (cellule : voir map:getInteractives())
local ok = map:useCell(ATELIER_CELL, -1)
if ok and craft:isOpen() then
    craft:putItem(INGREDIENT_1, 4)
    craft:putItem(INGREDIENT_2, 1)
    craft:changeQuantityToCraft(10)
    if craft:ready() then printSuccess("Craft réussi") end
    leaveDialog()
end
```

Copier

Module npc: — PNJ, banque & HDV (compat SnowBot)

Noms SnowBot. Les ids de PNJ sont des ids de MODÈLE (> 0, comme dans les scripts SnowBot) ou d'ACTEUR (négatifs, map:getNpcs()), résolus sur la carte courante ; 0 = premier vrai PNJ de la carte (jamais un indice de chasse au trésor) ; un modèle absent de la carte = PNJ introuvable (échec immédiat, npcBank compris). Voir aussi talkNpc / replyNpc (catégorie PNJ / dialogue).

reply(n) ≠ replyNpc(i)

npc:reply(n) suit SnowBot : n < 0 = n-ième réponse visible (-1 = la première), n > 0 = IDENTIFIANT de réponse (getReplies()). replyNpc(i) prend toujours un INDEX 1-based.

```
npc:npc(id [, action=3]) → bool
```

Bloquant

Action sur un PNJ puis attente de l'ouverture (≤ 4 s ; la question PNJ ≤ 1,5 s de plus) : 3 = parler, 5 = bascule du HDV en mode VENTE, 6 = bascule en mode ACHAT, autre = action PNJ. 5 / 6 envoient la MÊME requête que goToHdv / goToHdvBuy pour basculer le mode (NpcGenericActionRequest, acteur -1 = l'hôtelier) ; mode reconnu tout seul : sale:* marche ensuite comme après goToHdv / goToHdvBuy. Permission « Hôtel de Vente » requise : sans lui, false tout de suite, rien n'est envoyé et la console du script indique que la permission « Hôtel de Vente » est requise. En Dofus 3, l'hôtelier s'ouvre par son INTERACTIF (goToHdv / goToHdvBuy font les deux) : forme SnowBot = ouvre-le d'abord (map:door / goUseInteractive / map:useById), puis npc:npc(-1, 5 | 6). false + avertissement si le PNJ est introuvable (0 = premier vrai PNJ de la carte, jamais un indice de chasse) ou ne répond pas.

```
map:door(CELLULE_HOTELIER)
```

-- ouvre i

Copier

```
if npc:npc(-1, 5) then sale:SellItem(289, 100, 2500) end -- bascule en ven
```

`npc:npcBank([id]) → bool`

Bloquant

Perm.Gather

Ouvre la banque : id omis / 0 / -1 = banquier de la carte (reconnu par son modèle), id > 0 = modèle, id < -1 = acteur. Un modèle absent de la carte échoue tout de suite (false + « PNJ introuvable » dans la console), sans demande de banque envoyée. Le coffre reste ouvert (putAllItems, exchange:*, leaveDialog).

`npc:reply(n) → bool`

Bloquant

Répond (voir l'encadré) puis attend (≤ 5 s) : true = une NOUVELLE question est affichée ; false = dialogue fermé, délai écoulé ou réponse refusée.

```
if npc:npc(PNJ_ID) then
    for _, id in ipairs(npc:getReplies()) do print("réponse possible : " ..
        npc:reply(-1)           -- 1re réponse visible
        npc:leaveDialog()
    end
```

Copier

`npc:SendReply(id) → bool`

`npc:sendReply(id) → bool`

Envoie la réponse d'identifiant id sans attendre la suite.

`npc:getReplies() → number[]`

`npc:getQuestionId() → number`

`npc:question() → table|nil`

`npc:questionText() → string`

`npc:hasReply(id) → bool`

`npc:isDialogOpen() → bool`

Ids des réponses visibles ({} sans dialogue ; aussi getReplies() et global:getReplies()), id du message de la question (0), question complète { messageId, params, replies, text } ou nil,

texte de la question ("" si inconnu), cette réponse est-elle proposée ?, un dialogue PNJ est-il ouvert ?

`npc:leaveDialog()` → `bool`

= `global:leaveDialog()` : ferme le dialogue et laisse ~250 ms au serveur.

Module chat: — messages (compat SnowBot)

Noms et ORDRE d'arguments SnowBot. Permission « Réponse automatique » ; true si le message est parti.

Ordre des arguments

`chat:sendPrivateMessage(contenu, destinataire)` garde l'ordre SnowBot, sans correction automatique. Le global `sendPrivateMessage(pseudo, message)` garde l'ordre MizanBot.

`chat:sendPrivateMessage(contenu, destinataire)` → `bool`

Perm.PmRelay

MP à un joueur, par son pseudo.

```
chat:sendPrivateMessage("j'arrive", "SuperPote")
```

Copier

`chat:sendGeneralMessage(texte)` → `bool`

`chat:sendGuildMessage(texte)` → `bool`

`chat:sendAllianceMessage(texte)` → `bool`

`chat:sendTradeMessage(texte)` → `bool`

`chat:sendRecruitmentMessage(texte)` → `bool`

`chat:sendChannelMessage(canal, texte)` → `bool`

Perm.PmRelay

Canaux général (0), guilde (2), alliance (3), commerce (5), recrutement (6) ;
`sendChannelMessage` = canal au choix (= global `sendChannelMessage`).

Module character:

Identité du perso piloté, PV/énergie (alias SnowBot des getters globaux), caractéristiques et abonnement.

```
character:id() → number  
character:identifiant() → string  
characterId (nombre / chaîne).
```

```
character:name() → string  
character:level() → number  
character:breed() → number  
character:breedName() → string  
character:sex() → bool  
character:server() → string  
character:serverId() → number  
character:getStatus() → string
```

Nom, niveau, classe (id / nom), sexe (true = femme), serveur (nom / id), activité (= getStatus()). ⚠ breedName() rend le nom français ACCENTUÉ (« Féca », « Xélor », « Crâ ») là où SnowBot rendait l'identifiant sans accent (« Feca ») : dans un script porté, compare plutôt breed().

```
character:lifePoints() → number  
character:maxLifePoints() → number  
character:lifePointsP() → number  
character:energyPoints() → number  
character:maxEnergyPoints() → number  
character:experience() → number
```

PV, PV max, % PV, énergie, énergie max et XP totale (-1 si inconnus).

```
character:statPoints() → number  
Points de caractéristiques restants à investir.
```

```
character:upgradeStat(nom, n) → ok, restants
```

```
character:UpgradeVitality(n) → bool
character:UpgradeWisdom(n) → bool
character:UpgradeStrength(n) → bool
character:UpgradeStrenght(n) → bool
character:UpgradeIntelligence(n) → bool
character:UpgradeChance(n) → bool
character:UpgradeAgility(n) → bool
```

Bloquant

Perm.Fight

Investit n points dans vitality, wisdom, strength, intelligence, chance ou agility. Les alias Upgrade* (SnowBot) ne renvoient que ok.

```
local ok, reste = character:upgradeStat("vitality", character:statPoints())
```

Copier

```
character:resetStats() → bool
resetStats() → bool
```

Perm.Fight

Réinitialise les caractéristiques (hors combat).

```
character:serverName() → string
character:isInFight() → bool
character:kamas() → number
character:statsPoint() → number
```

Noms SnowBot : = getServerName, isInFight, getKamas (-1 si inconnu), statPoints.

```
character:getVitalityBase() → number
character:getWisdomBase() → number
character:getStrengthBase() → number
character:getStrenghtBase() → number
character:getIntelligenceBase() → number
character:getChanceBase() → number
character:getAgilityBase() → number
```

Caractéristiques de BASE (points investis, hors équipement), lues dans la dernière fiche de

caractéristiques reçue ; -1 si inconnue. getStrenghtBase = orthographe SnowBot.

`character:getStats()` → `table|nil`

Dernière fiche de caractéristiques (message CharacterCharacteristicsEvent) en table de paquet avec défauts proto3 (façon SnowBot : champ absent du message racine = 0 / false / "") ; snake_case, camelCase toléré ; nil si jamais reçue.

`character:getSecondsBeforeFreeMode()` → `number`

`character:freeMode()` → `bool|nil`

Secondes avant la fin de l'abonnement (0 si terminé, -1 si inconnu) / true si le compte n'est plus abonné (nil si inconnu).

`character:getCostVitality()` → `-1`

`character:getSpellMinPlayerLevel()` → `-1`

`character:nickname()` → `" "`

`character:accountId()` → `-1`

`character:trusted()` → `false`

`character:certified()` → `false`

`character:getURLFaceSkin()` → `" "`

`character:getURLBodySkin()` → `" "`

Valeurs neutres (données non extraites ou sans équivalent) : getCostVitality / Wisdom / Strenght / Intelligence / Chance / Agility → -1 (avertissement), getSpellMinPlayerLevel → -1, nickname → "", accountId → -1, trusted / certified → false, getURLFaceSkin / getURLBodySkin → "".

`character:askShieldCode()` → `false`

`character:sendShieldCode()` → `false`

`character:askShieldCodeNew()` → `false`

`character:sendShieldCodeNew()` → `false`

REFUSÉS (sécurité du compte) : renvoient false et journalisent un avertissement. Le code du Bouclier ne passe jamais par un script.

Module mount: — montures (valeurs neutres)

Présent pour que les scripts d'élevage SnowBot se CHARGENT et tournent : les messages de monture ne sont pas capturés, donc rien n'est envoyé. Chaque verbe journalise un avertissement (une fois) et rend une valeur neutre.

```
mount:hasMount() → false
mount:isRiding() → false
mount:toggleRiding() → false
mount:feedMount() → false
mount:setXpRatio(ratio) → false
mount:switchMount() → false
```

false : hasMount, isRiding, isRideable, toggleRiding, setXpRatio, feedMount, mountInformation, switchMount, RetireMountSet, UpdateCertificate, DeleteMountList, movePaddockItem, addPaddockItem, putPaddockItem, removePaddockItem.

```
mount:level() → -1
mount:getEnergy() → -1
mount:myMount() → nil
mount:moveMounts() → 0
mount:inventoryMounts() → {}
```

-1 : level, getEnergy, getMaxEnergy, id, maxPods ; nil : myMount ; 0 : moveMounts ; {} : inventoryMounts, inventoryMountsData, stabledMounts, paddockedMounts, newBorns, getPaddockInformations. Champs publics SnowBot (valeurs, pas des fonctions) : TiredMount = 210 ; BankReturn, LevelUp, BoucleNumber = 0 ; CellsObjects, IdsBabyWorn, MountSet, MountInventory, MountEtable, MountEnclos, NewBornsData, guidWaitCertificat, paddockInformations = {} — utilisables comme état par le script.

Module stats: — compteurs

```
stats:harvests() → number
stats:resources() → number
stats:fights() → number
stats:won() → number
stats:lost() → number
```

Compteurs de la SESSION du script : récoltes, ressources récoltées, combats, victoires, défaites.

`stats:reset()`

Remet la session à zéro (pas l'historique de l'onglet Stats).

`stats:summary([période]) → table`

Historique persisté : { harvests, resources, fights, won, lost, deaths, kamas, xp, bankVisits, bottingSec }; période "today" (défaut), "day", "week", "month". {} tant que le perso n'est pas identifié.

```
local s = stats:summary("week")
print("Semaine : " .. (s.kamas or 0) .. " k, " .. (s.fights or 0) .. " comba
```

Copier

Module quest: — quêtes

nil = inconnu

nil signifie « inconnu » (quête pas en cours ou objectifs non détaillés), jamais « aucun objectif ». reached = true veut dire FAIT (inverse de l'ancien objectiveStatus de SnowBot / Dofus 2).

`quest:loaded() → bool`

true quand la liste des quêtes a été reçue (non bloquant).

`quest:initialize() → bool`

Bloquant

Si la liste des quêtes de la connexion manque encore, l'attend (≤ 3 s, une seule fois par run ; les appels suivants répondent tout de suite), puis renvoie loaded() (SnowBot la demandait et bloquait).

`quest:isFinished(id) → bool`

`quest:finished() → number[]`

Quête terminée ? / ids des quêtes terminées.

```
quest:active() → table[]  
quest:activeQuests() → number[]  
quest:isActive(id) → bool  
quest:questActive(id) → bool
```

Quêtes en cours { id, step, name } (step -1 si inconnue) / leurs ids / une quête est-elle en cours ?

```
quest:currentStep(id) → number  
quest:questCurrentStep(id) → number
```

Étape courante (-1 si inconnue ; demande l'information au serveur si besoin).

```
quest:objectives(id) → table[]  
quest:doneObjectives(id) → number[]|nil  
quest:questDoneObjectives(id) → number[]|nil  
quest:remainingObjectives(id) → number[]|nil  
quest:questRemainingObjectives(id) → number[]|nil  
quest:questObjectiveDone(questId, objectiveId) → bool
```

Objectifs { id, reached, current, max } / ids faits / ids restants / un objectif est-il fait ?

```
quest:refresh(id [, timeoutMs=3000]) → bool
```

Bloquant

Redemande au serveur l'état de la quête id et attend la réponse.

```
quest:validateObjective(questId, objectiveId [, timeoutMs=5000]) → bool
```

Bloquant

Perm.Gather

Valide un objectif de quête et attend la confirmation.

```
quest:name(id) → string
```

```
quest:stepName(stepId) → string
quest:npcName(id) → string
quest:monsterName(id) → string
```

Noms clairs : quête, étape, PNJ, monstre.

```
if quest:isActive(QUETE_ID) then
    print(quest:name(QUETE_ID) .. " : étape " .. quest:currentStep(QUETE_ID)
end
```

Copier

```
quest:text(id) → string
quest:challengeName(id) → string
quest:objectiveTypeName(id) → string
quest:stepDescription(stepId) → string
quest:objectiveText(id) → string
quest:path() → nil
quest:mainPath() → string
```

Textes du jeu ("" si inconnus) : texte par id, nom de défi, type d'objectif, description d'étape. objectiveText rend toujours "" (pas de table fiable côté Dofus 3) ; path() → nil et mainPath() → "" (chemins du pack de quêtes SnowBot, sans objet ici).

Module treasure: — chasse au trésor (compat SnowBot)

Chasse au trésor : état décodé en direct, drapeaux, creusage et abandon. Chaque action attend la réponse du serveur, au plus 30 s (SnowBot : 300 s), sous le seuil anti-blocage. Actions gâtées par la permission « Récolte auto ». L'état de chasse est oublié quand un autre perso est sélectionné en jeu.

Index d'étape 0-based

Comme chez SnowBot, les verbes qui prennent un index d'étape (stepType, getNextFlagDirection, getNextFlagName, getNextFlagId) comptent à partir de 0 ; -1 ou omis = la DERNIÈRE étape connue (l'indice en cours). C'est une exception aux listes 1-based de l'API.

Pas de résolveur d'indices

treasure.nextMapId vaut toujours 0 : MizanBot n'interroge aucun site tiers pour trouver la carte d'un indice (le résolveur de SnowBot n'est pas porté). Le script cherche l'indice lui-même (carte par carte dans getNextFlagDirection(), map:getMap()...) ou via un service de son choix

(developper:getRequest, permission « Réponse automatique »).

treasure:startHunt(niveau) → number

Bloquant

Perm.Gather

Démarre une chasse avec le coffre de chasse de la carte courante. niveau = niveau de chasse voulu, converti en choix du coffre par tranches de 20 niveaux (ton niveau plafonné à 200) ; 0 = le premier choix proposé ; niveau au-dessus de ta tranche → 0 sans rien envoyer (comme SnowBot). Attend la réponse (≤ 12 s). Renvoie 1 démarrée, 5 déjà en cours, autre valeur > 0 = code de refus rendu par le serveur, 0 échec (pas de coffre de chasse sur la carte, droit manquant, pas de réponse), $-(\text{minutes} + 1)$ = délai avant de pouvoir en lancer une nouvelle.

treasure:huntFlag() → bool

treasure:huntClear() → bool

Bloquant

Perm.Gather

huntFlag pose le drapeau de l'étape en cours sur la carte courante et attend la mise à jour (≤ 30 s ; true = posé, false = refusé, sans réponse ou hors chasse). huntClear retire les drapeaux posés (à partir du premier) et attend la mise à jour (≤ 2 s) ; true aussi s'il n'y en avait aucun ou hors chasse.

treasure:huntDig() → bool

treasure:huntDigTry() → bool

treasure:lastDigResult() → number

Bloquant

Perm.Gather

Creuse : true = étape validée. treasure.validated (sens SnowBot) = 1 si le dernier creusage a réussi, 0 pour tout autre résultat (et avant tout creusage), -1 tant qu'aucune réponse n'est arrivée (sans réponse, ou étape de combat) ; le code brut du serveur reste lisible via lastDigResult(). Si l'étape finale est un combat : true tout de suite (le combat démarre). huntDigTry : comme huntDig, et true aussi si le dernier drapeau, posé sur cette carte, est déjà validé. lastDigResult = dernier résultat brut (-1 si aucun).

treasure:isHunting() → bool

treasure:isReadyToDig() → bool

treasure:getStepCount() → number

`treasure:getStartMapId()` → `number`

`treasure:state()` → `table|nil`

Chasse en cours ? / prêt à creuser (tous les drapeaux posés, ou combat final ; aussi sous le nom SnowBot FuAJ2GhY5548587) / nombre total d'étapes (0 hors chasse) / carte de départ de l'étape = carte du dernier drapeau, sinon départ de la chasse (0 hors chasse) / état complet { questType, startMapId, totalStepCount, checkPointCurrent, checkPointTotal, availableRetryCount, flags = {{mapId, state}}, knownStepsList = {{type, direction, npcId?, poiLabelId?, mapCount?}} } ou nil (hors chasse) ; alias currentCheckPoint, totalCheckPoint et knownSteps.

`treasure:stepType([i])` → `string`

`treasure:isHintStep()` → `bool`

`treasure:isPhorreurStep()` → `bool`

`treasure:isFightStep()` → `bool`

`treasure:isPhorreurInMap()` → `bool`

Type d'étape : "poi" (indice à trouver), "hint" (phorreur), "direction", "fight", "dig" ("" si inconnue). isHintStep = indice, isPhorreurStep = phorreur, isFightStep = combat final (étape en cours) ; isPhorreurInMap = le phorreur de l'étape en cours est sur la carte.

`treasure:getNextFlagDirection([i])` → `string`

`treasure:getNextFlagName([i])` → `string`

`treasure:GetNextFlagName([i])` → `string`

`treasure:getNextFlagId([i])` → `number`

`treasure:getPos(mapId)` → `string`

Direction de l'étape ("right", "bottom", "left", "top" ; "5" si la direction reçue n'est aucune des quatre, comme SnowBot ; "" pour une étape sans direction), nom de l'indice ou du phorreur (""), id du libellé d'indice ou du PNJ phorreur (-1), coordonnées d'une carte au format SnowBot "[x,y]" ("" si inconnues).

`treasure:quit()` → `bool`

`treasure:quitNow()` → `bool`

`treasure:isGivingUp()` → `bool`

`treasure:giveUpCooldown()` → `number`

Bloquant

Perm.Gather

quit abandonne et attend (≤ 5 s) ; giveUpCooldown() repart de 0 à chaque quit lancé en chasse (jamais de délai périmé d'un refus précédent) : true = chasse terminée (true aussi hors chasse, comme quitNow) ; si le serveur répond par un message système portant un délai (1er paramètre numérique), false et giveUpCooldown() = ce délai + 1 min (SnowBot se déconnectait N + 1 min : ici rien n'est forcé, à toi d'attendre ou d'appeler global:reconnectBis) ; autre message ou pas de réponse : false (giveUpCooldown() = 0). quitNow envoie l'abandon sans attendre. isGivingUp = un abandon est en cours.

```
treasure:lastHuntReport() → {}  
treasure:sendLastHuntReport()  
treasure:printLastHuntReport()
```

Champs publics SnowBot lisibles en direct : treasure.treasure (= state()), _isStarted / _isHunting (chasse en cours), giveUp, nextMapId (toujours 0), validated (1 / 0 / -1, voir huntDig), treasureHuntRequestAnswerMessage = { result }, giveUpRequest = { questType }. Rapports : lastHuntReport → {}, sendLastHuntReport / printLastHuntReport / treasureInactivity → sans effet ; gdPggXxViU (résolveur d'indices de SnowBot, non porté) → nil + avertissement (une fois).

```
-- Squelette de chasse (la recherche de la carte de l'indice reste à écrire)  
function move()  
    if not treasure:isHunting() then  
        local r = treasure:startHunt(0)  
        if r < 0 then printWarning("Nouvelle chasse dans " .. -r .. " min") delay  
        return  
    end  
    if treasure:isReadyToDig() then treasure:huntDig() return end  
    printMessage("Indice : " .. treasure:getNextFlagName() .. " vers " .. treasure:  
    -- ... avancer carte par carte dans cette direction jusqu'à l'indice, puis :  
    -- treasure:huntFlag()  
end
```

Copier

Module data: (alias d2data:) — données statiques du jeu

Un script qui assigne un global data = ... masque ce module pour lui-même : utilise local data.

```
data:monsterName(id) → string  
data:npcName(id) → string
```

```
data:itemName(gid) → string
data:spellName(id) → string
data:jobName(id) → string
data:questName(id) → string
data:subAreaName(id) → string
data:areaName(id) → string
```

Noms clairs tirés des données du jeu (lectures pures).

```
print(data:spellName(161))
```

Copier

```
data:itemLevel(gid) → number
data:itemWeight(gid) → number
```

Niveau / poids d'un objet.

```
data:text(id) → string
data:challengeName(id) → string
data:poiName(id) → string
```

Texte du jeu par id, nom d'un défi, nom d'un indice de chasse au trésor ; "" si inconnu.

```
data:mapData([mapId]) → table
```

{ mapId, x, y, subAreaId, areaId } d'une carte (carte courante si omis ; x / y absents si inconnus, areaId -1).

```
local m = data:mapData()
print(m.x .. "," .. m.y .. " sous-zone " .. m.subAreaId)
```

Copier

Module d2data: (compat SnowBot)

d2data: est la MÊME table que data: : tous ses verbes (noms, niveaux, poids, text, challengeName, poiName, mapData) répondent aussi sous d2data:. Ce qui dépend des fichiers D2O de Dofus 2 n'a pas d'équivalent.

```
d2data:itemName(gid) → string
d2data:mapData([mapId]) → table
```

Exemples : ce sont les verbes de data:, sous le nom SnowBot.

```
print(d2data:itemName(289), d2data:mapData().subAreaId)
```

Copier

```
d2data:objectFromD2O() → nil
d2data:allObjectsFromD2O() → {}
```

Pas d'accès générique aux tables D2O (format Dofus 2) : nil / {} + avertissement (une fois).
Utilise les lectures nommées (itemName, monsterName, mapData...).

```
d2data:exportD2O() → false
d2data:exportAllD2O() → false
d2data:exportMapData() → false
d2data:exportAllMapData() → false
```

REFUSÉS (bac à sable : un script n'écrit aucun fichier de données sur disque) : false + avertissement.

Modules memory: & sharedMemory: — état persistant

Stockage clé/valeur persistant (valeurs converties en chaînes) dans
%APPDATA%/MizanBot/memory/. memory: est propre au personnage, sharedMemory: est
commun à tous les bots (coordination chef ↔ mules, reprise après crash).

memory: utilisé au top-level (avant l'identification du perso) écrit dans un compartiment "?" :
utilise-le dans move(). Les nombres se relisent avec tonumber().

Mémoire SnowBot

global:addInMemory... (par personnage ET par serveur de jeu, conservée d'un rechargement à
l'autre et à travers reconnect* et PLANNING), *GlobalMemory et
snowbotController:addInMemory... sont une mémoire RAM TYPÉE (nombres, booléens, tables)
mais perdue à la fermeture de l'app ; memory: / sharedMemory: sont persistés mais en
CHAÎNES. Voir la table global:.

```
memory:set(clé, valeur) → true
sharedMemory:set(clé, valeur) → true
```

Enregistre valeur (convertie en chaîne) sous clé.

```
memory:get(clé [, défaut]) → string
sharedMemory:get(clé [, défaut]) → string
```

Valeur de clé, sinon défaut (nil si omis).

```
memory:delete(clé) → bool
memory:all() → table
sharedMemory:delete(clé) → bool
sharedMemory:all() → table
```

Supprime une clé / renvoie toute la table clé → valeur.

```
local n = tonumber(memory:get("banques", "0")) + 1
memory:set("banques", n)
sharedMemory:set("spot_libre", getMapId())
```

Copier

Table global: — verbes SnowBot

La table global: des scripts SnowBot : alias des verbes globaux MizanBot (appel colon ou point), mémoire RAM typée, équipe, compte (déconnexion / reconnexion), console et dossiers virtuels. Elle expose aussi lastModeratorMessage, setPrivate, setPrivateTeam, getPlayerStatus, isPrivate (voir Messagerie) et getReplies (voir PNJ / dialogue).

Mémoire RAM typée ≠ memory:

addInMemory & co gardent le TYPE (nombre, booléen, chaîne, table — une fonction n'est pas stockable) en RAM, jusqu'à la fermeture de l'app. global:addInMemory... est propre au PERSONNAGE sur son SERVEUR de jeu (clé = serveur + nom du perso : deux persos homonymes sur deux serveurs ne partagent rien ; avant l'identification du perso, compartiment provisoire du bot) : conservée d'un rechargement de script à l'autre ET à travers global:reconnect*, PLANNING et une relance après crash. Une table rangée est rendue telle quelle (la MÊME table) au run qui l'a rangée : table.insert(global:remember("k"), v) persiste, comme chez SnowBot ; un autre script (autre bot, /cmd, script rechargé) en reçoit une copie, qu'il adopte comme table vivante une fois le run d'origine terminé. memory: / sharedMemory:

sont PERSISTÉS sur disque mais en chaînes. addInMemory n'écrase jamais une clé existante (utilise editInMemory) ; editInMemory n'agit que si la clé existe.

Identifiants jamais exposés

username() / getAlias() renvoient l'ALIAS du compte (libellé de l'onglet Comptes, sinon nom du perso, sinon « Compte N »), jamais l'identifiant de connexion. password() et getToken() sont REFUSÉS : ils renvoient "" et journalisent « refusé par sécurité » (une fois).

```
global:printMessage(message [, niveau])
global:printSuccess(message)
global:printWarning(message)
global:printError(message)
global:printColor(message, couleur)
global:delay(ms)
global:isScriptPlaying() → bool
global:isInFight() → bool
global:afterFight() → bool
global:getCountFight() → number
global:getCountGather() → number
global:resetCountFight()
global:resetCountGather()
global:joinBoss() → bool
global:inviteTeam() → number
global:isInParty([nom | id]) → bool
global:reloadScript() → bool
global:getCharacterName() → string
global:getCharacterLevel() → number
global:getKamas() → number
global:getMapId() → number
global:getCurrentCell() → number
global:sendWebhook(url, message) → bool
global:notify(message [, son]) → bool
```

Mêmes verbes que les globaux homonymes (mêmes retours, droits et blocages).
RestartScript, RestartScriptThread, reloadScriptInTheNewThread et
reloadScriptInTheNewThreadWithFile = reloadScript.

`global:finishScript()`

Arrête définitivement le script (= stopScript ; stopped("script") est quand même appelé).

`global:leaveDialog()` → `bool`

Ferme le dialogue / l'échange courant puis laisse ~250 ms au serveur (SnowBot attendait la fermeture).

`global:loadAndStart(chemin [, casser])` → `bool`

`global:LoadAndStartThread(chemin)` → `bool`

L'argument est d'abord essayé TEL QUEL, comme loadScript (un nom de la bibliothèque du planning peut contenir « / » ou « \ ») ; sinon il est réduit à son nom de fichier (C:\...\ble.lua → ble), résolu dans la bibliothèque du planning puis dans %APPDATA%/MizanBot/scripts. Un chemin disque manifeste (lecteur, séparateur initial, ./, ../, scripts\... de getCurrentScriptDirectory()) est réduit d'emblée. Aucun chemin disque n'est lu, 2e argument ignoré. Aussi en global plat loadAndStart.

`global:loadAndStart(global:getCurrentScriptDirectory() .. "\\farm_50.104`

Copier

`global:addInMemory(clé, valeur)`

`global:editInMemory(clé, valeur)`

`global:remember(clé)` → `valeur|nil`

`global:deleteMemory(clé)` → `ancienne valeur`

`global:deleteAllMemory()`

`global:AddInGlobalMemory(clé, valeur)`

`global:EditInGlobalMemory(clé, valeur)`

`global:GetInGlobalMemory(clé)` → `valeur|nil`

`global:DeleteFromGlobalMemory(clé)` → `ancienne valeur`

`global:DeleteAllGlobalMemory()`

Mémoire RAM typée (voir l'encadré). Les verbes en minuscule sont PAR PERSONNAGE et par serveur de jeu (conservés aux rechargements et reconnexions ; deux homonymes sur deux serveurs ne partagent rien) ; les verbes *GlobalMemory sont COMMUNS à tous les bots de l'app. remember / GetInGlobalMemory rendent nil si la clé est absente ; deleteMemory /

DeleteFromGlobalMemory rendent l'ancienne valeur. Tables : 8 niveaux, 10 000 éléments max ; le run qui a rangé une table la relit telle quelle (mutations en place conservées), les autres scripts en reçoivent une copie.

```
global:addInMemory("passages", 0) -- ignoré si la clé existe
global:editInMemory("passages", global:remember("passages") + 1)
global:AddInGlobalMemory("spot", { map = getMapId(), libre = true })
global:addInMemory("vus", {})
table.insert(global:remember("vus"), getMapId()) -- même table : l'ajout p
```

```
global:isBoss() → bool
global:getTeamMembers() → string[]
global:getTeamMembersNames() → string[]
global:getTeamMembersIds() → number[]
global:getTeamMembersUsernames() → string[]
global:isInTeam(nom) → bool
```

Chef d'équipe ? / membres (chef en premier) : noms des persos, characterIds, ALIAS des comptes — listes VIDES si tu n'es pas le chef (comme SnowBot). isInTeam(nom) : nom de perso, alias ou compte (chef seulement).

```
global:username() → string
global:getAlias() → string
global:accountTag() → string
global:charactersList() → table[]
```

Alias du compte (jamais l'identifiant de connexion) ; accountTag → "" (pseudo#tag non capturé) ; charactersList → { { name, id, level } } du SEUL perso connecté.

```
global:disconnect() → bool
global:reconnect(heures) → bool
global:reconnectBis(minutes) → bool
global:reconnectSec(secondes) → bool
global:reconnectWithToken(secondes) → bool
```

Déconnecte ce bot (chef : toute son équipe) par l'onglet Bots — le script s'arrête avec lui. reconnect* le relance après le délai (48 h max) avec le même perso et REPREND le même

script une fois la carte chargée (5 min max après la relance). true = demande enregistrée (exécutée au tick suivant de l'app). Mêmes gardes que le planning : licence, permission « Multi » si un autre compte tourne, Dofus.exe connu en mode MITM. Une reconnexion programmée est ANNULÉE si, avant l'échéance, le compte est arrêté ou lancé par un autre chemin (Arrêter, Lancer, Lancer tout, Connecter maintenant, « Connecter en jeu » de l'équipe, loadAnAccount d'un autre script) ou s'il est déjà en ligne à l'échéance : le script n'est alors pas repris. D'ici l'échéance, la connexion auto du planning de l'onglet ne relance pas le compte. disconnect() sur un compte géré par cette connexion auto = comme un arrêt manuel : pas de relance avant la prochaine occurrence de sa plage.

```
if global:elapsedTime() >= 4 then global:reconnect(1) end -- après Copier 1
```

```
global:random(min, max) → number  
global:elapsedTime() → number  
global:fightMessage([n]) → table|nil
```

Entier aléatoire inclusif (bornes permutées si inversées) ; heures ENTIÈRES depuis le 1er lancement d'un script sur ce bot (0 avant ; repart de zéro quand le bot est relancé, reconnect* compris) ; bilan du DERNIER combat (= combat:lastFightResult() + fightCounter (= getCountFight()) et duration serveur ; n ignoré), nil si aucun.

```
global:consoleLines() → string[]  
global:clearConsole()  
global:playAudio([fichier])  
global:exportAllPublicMethods() → string
```

300 dernières lignes de console de ce bot / vide cette mémoire (pas l'affichage de l'onglet) ; playAudio joue le son de notification de l'app (fichier ignoré) ; exportAllPublicMethods liste les verbes disponibles (« table:verbe() » et globaux).

```
global:getCurrentScriptDirectory() → string  
global:getCurrentDirectory() → string  
global:getAllFileNameInDirectory(dossier, ext) → string[]
```

Dossiers VIRTUELS : les deux premiers renvoient "scripts" (jamais un vrai chemin, qui contiendrait ton nom d'utilisateur Windows) ; getAllFileNameInDirectory rend les noms des fichiers *.ext (lua par défaut) de %APPDATA%/MizanBot/scripts (dossier ignoré ; fichiers directement dans scripts/ seulement, 500 max) ; ext = 1 à 10 lettres ou chiffres ASCII (« * » ou

« . » initial ignoré), sinon {}. Avec loadAndStart / dofile, l'idiome SnowBot « charger un fichier du dossier du script » marche.

```
global: thisAccountController() → table
global: getController() → table
global: leaderAccountController() → table|nil
```

Contrôleur de compte SnowBot, en table : { name, alias, pid, isLeader, mapId(), isInFight(), executeLua(code) }. executeLua exécute le code UNE fois sur ce membre (= team:runOnce, permission « Multi ») : false + avertissement sur soi-même ou sur un compte qui n'est pas un membre en jeu de ton équipe. leaderAccountController() = nil hors équipe.

```
local chef = global:leaderAccountController()
if chef and not chef.isInFight() then print("chef sur la carte " .. chef.map
```

Copier

```
global: askQuestion() → false
global: askInput() → ""
global: isKoliServer() → false
global: checkModerator() → false
global: openMitm() → false
global: password() → ""
global: getToken() → ""
```

Sans équivalent MizanBot : avertissement (une fois) et valeur neutre — false pour askQuestion, rcAskFight, rcAskGather, isKoliServer, isStuffForKoli, isRegisteredInKoli, unregisterFromKoliIfRegistered, editAlias, loadConfiguration, LoadConfigurationWithoutScript, checkModerator, antiModeratorActivated, groupCharacters, ungroupCharacters, openMitm ; nil pour blabla et activateAntiModerator ; "" pour askInput. password() et getToken() sont REFUSÉS ("").

Module config: — réglages de script (compat SnowBot)

Setters / getters SnowBot adossés aux globals de Configuration : un set* écrit le global (il vaut donc aussi pour fight(), gatherAll() et les étapes du trajet), un get* relit sa valeur. Appel colon ou point.

Différences

SnowBot réécrivait ces réglages avant chaque move() : ici un set* reste posé jusqu'au prochain set* ou à la fin du run (inutile de le rappeler à chaque tour). Les set*Monsters n'acceptent que des ids numériques (les noms restent possibles directement dans FORBIDDEN_MONSTERS / FORCE_MONSTERS, et les getters les ignorent). getMaxMonsters() rend 999 si MAX_MONSTERS est absent : la valeur réellement appliquée, aucun maximum (SnowBot : 8). getGatherList() lit ELEMENTS_TO_GATHER, sinon GATHER (même repli que le moteur).

```
config:setGatherList(liste | id, ...)
```

```
config:getGatherList() → number[]
```

= ELEMENTS_TO_GATHER ; getGatherList relit GATHER quand ELEMENTS_TO_GATHER n'est pas une table.

```
config:setMandatoryMonsters(liste)
```

```
config:getMandatoryMonsters() → number[]
```

```
config:setForbiddenMonsters(liste)
```

```
config:getForbiddenMonsters() → number[]
```

= FORCE_MONSTERS / FORBIDDEN_MONSTERS (ids numériques).

```
config:setMinMonsters(n)
```

```
config:getMinMonsters() → number
```

```
config:setMaxMonsters(n)
```

```
config:getMaxMonsters() → number
```

= MIN_MONSTERS / MAX_MONSTERS, bornés à 1-8 par les setters. Global absent : getMinMonsters() = 1, getMaxMonsters() = 999 (valeurs appliquées par le moteur).

```
config:setAmountOfSpecificMonsters({gid, min, max}, ...)
```

```
config:getAmountOfSpecificMonsters() → table
```

= MONSTERS_AMOUNT : pour chaque gid, un groupe qui en contient est écarté si son nombre est hors [min, max] (max omis = 8).

```
config:setGatherList({ 38 })
```

```
config:setForbiddenMonsters({ MONSTRE_INTERDIT })
```

```
config:setMaxMonsters(4)
```

-- = ELEMENTS_TO_GATHER Copier

```
config:setAmountOfSpecificMonsters({ { MONSTRE_ID, 1, 2 } }) -- 1 ou 2 de ce mc
```

Module snowbotController: — comptes de l'app (compat SnowBot)

Vue « contrôleur » SnowBot des comptes enregistrés dans l'onglet Comptes, désignés par leur ALIAS (libellé, sinon nom du perso, sinon « Compte N » = rang dans l'onglet Comptes). Un nom saisi par le script peut être un alias ou un nom de perso. MizanBot n'a pas de script « contrôleur » sans compte : ces verbes s'utilisent depuis le script d'un bot.

Refusés (sécurité)

getPassword → "" ; registerAnAccount, createCharacter, assignProxyToAnAccount, unlockShield, addPushBulletKey → false. Chacun journalise « refusé par sécurité » (une fois) : création de compte ou de perso, proxys, Bouclier et mots de passe ne sont jamais pilotés par un script. activateAntiModerator, loadTeamAccount, loadTeamAccountbak et loadControllerAccount → false aussi, avec l'avertissement « sans équivalent dans MizanBot ».

```
snowbotController:printMessage(message)
snowbotController:printSuccess(message)
snowbotController:printWarning(message)
snowbotController:printError(message)
snowbotController:delay(ms)
snowbotController:version() → "PC"
```

Journal et pause (= globaux) ; version() rend "PC".

```
snowbotController:addInMemory(clé, valeur)
snowbotController:editInMemory(clé, valeur)
snowbotController:remember(clé) → valeur|nil
snowbotController:deleteMemory(clé) → ancienne valeur
snowbotController:deleteAllMemory()
```

Mémoire RAM typée commune à TOUTE l'app (espace distinct de global:*GlobalMemory), mêmes règles que global:addInMemory (type gardé, même table pour le run qui l'a rangée, copie pour les autres), perdue à la fermeture de l'app.

```
snowbotController:getRequest(url [, noms, valeurs]) → string
snowbotController:postRequest(url, data [, noms, valeurs]) → string
```

= developer:getRequest / postRequest (module http:).

snowbotController:**getLoadedAccounts**() → table[]

snowbotController:**getAccount**(nom) → table|nil

Contrôleurs { alias, name, pid, loaded, mapId(), isInFight(), executeLUA(code) } des comptes lancés / d'un compte enregistré (nil si inconnu). executeLUA = exécution unique sur ce compte (= team:runOnce, permission « Multi ») : false + avertissement sur soi-même, sur un compte non lancé ou sur un compte qui n'est pas un membre en jeu de TON équipe MizanBot.

```
for _, acc in ipairs(snowbotController:getLoadedAccounts()) do
    print(acc.alias .. " : carte " .. acc.mapId())
end
```

[Copier](#)

snowbotController:**getAliasLoadedAccounts**() → string[]

snowbotController:**getAliasNotLoadedAccounts**() → string[]

snowbotController:**getAliasAllRegisteredAccounts**() → string[]

snowbotController:**accountIsLoaded**(nom) → bool

snowbotController:**accountIsRegistered**(nom) → bool

Listes d'ALIAS des comptes lancés / non lancés / tous, et tests par nom. Variantes acceptées : getUsernameLoadedAccounts, getUsernameOfLoadedAccounts, getAliasOfLoadedAccounts, getUsernameNotLoadedAccounts, getUsernameAccountNotLoaded, getUsernameAllRegisteredAccounts — les variantes « Username » renvoient AUSSI des alias. Instantané rafraîchi environ toutes les 1,5 s.

snowbotController:**loadAnAccount**(nom [, connexionAuto]) → bool

snowbotController:**unloadAnAccount**(nom) → bool

Lance / arrête un compte ENREGISTRÉ par l'onglet Bots, avec les gardes du planning (licence, permission « Multi » si un autre compte tourne déjà, Dofus.exe connu en mode MITM). Le compte est lancé avec son perso habituel ; connexionAuto est ignoré. true = demande enregistrée (exécutée au tick suivant de l'app ; sans effet si le compte est déjà lancé / déjà arrêté) ; false + avertissement si le nom est inconnu. unloadAnAccount sur un compte géré par la connexion auto du planning = comme un arrêt manuel : pas de relance avant la prochaine occurrence de sa plage.

```
if not snowbotController:accountIsLoaded("Mule-Bob") then
    snowbotController:loadAnAccount("Mule-Bob")
end
```

[Copier](#)

Plateforme : hooks de paquets, modules, fichiers & bascule de script

Table de paquet

Champs par nom (snake_case), sous-messages en sous-tables, listes 1-based, plus __name, __dir ("in"/"out") et __truncated / __error éventuels. Les valeurs par défaut (0, false, "") sont ABSENTES des données (pairs() ne les voit pas) : avec registerMessage / waitFor, un champ absent vaut nil. Les consommateurs façon SnowBot (developer:registerMessage, waitForAny, lastMessage, historicalMessage, character:getStats) rendent à la place la valeur par défaut d'un champ connu du message racine (0, false, "", {} pour une liste, nil pour un sous-message — défauts proto3, comme les objets SnowBot). Noms camelCase / PascalCase acceptés partout (msg.messageId, msg.MessageId → message_id). t.Count = nombre d'éléments d'une LISTE ; sur un message qui a un vrai champ count, t.Count rend ce champ. Les champs non nommés apparaissent en f<n>. Les gestionnaires tournent sur le thread du script aux points sûrs (delay, boucle principale, attente de fight(), waitFor) ; 5 erreurs consécutives désactivent un gestionnaire ; file de 256 paquets.

Noms de messages (Dofus 3 et Dofus 2)

Partout où un nom de message est attendu (registerMessage, waitFor, waitForAny, lastMessage, developer:suspendScriptUntil...) : nom Dofus 3 du catalogue, nom Dofus 2 traduit par la table d'alias SnowBot (ex. NpcDialogQuestionMessage → NpcDialogQuestionEvent, GameFightEndMessage → FightEndEvent), repli « XxxMessage » → XxxEvent / XxxRequest / XxxResponse / Xxx, ou identifiant PUBLIC Dofus 2 (nombre). La table d'alias ne vise que des messages du catalogue Dofus 3 courant : un nom Dofus 2 sans équivalent actuel (montures, aperçu d'accessoire...) reste « message inconnu ». Les hooks OBSERVENT seulement : un paquet sortant n'est jamais retenu ni modifié (SnowBot permettait d'intercepter puis de réémettre).

Messages sensibles refusés

Les messages qui portent un secret de session ou une donnée de compte (jetons de session, de reconnexion ou de boutique, clés d'API, tickets d'authentification, identification et identifiants, cert Bouclier, codes de serrure de maison ou de coffre, secret de suppression de personnage, e-mail du compte, identifiant et défi de connexion du client, et tout message dont le nom évoque un jeton, un ticket, une clé d'API, une identification ou une authentification, des

identifiants, un certificat, un mot de passe, un code de sécurité ou un secret) ne sont JAMAIS accessibles à Lua, sous aucun nom ni identifiant : `registerMessage` (global et `developer:`), `unRegisterMessage` → `false` ; `waitFor` → `nil` ; `waitForAny` les ignore (aucun nom valide : `nil`) ; `suspendScriptUntil*` attend tout le délai puis `false` / `"False"` ; `lastMessage` → `nil` ; `historicalMessage` → `{}` ; `createMessage` → `nil`. Un avertissement « refusé par sécurité » est journalisé (une fois). Ils ne sont jamais mémorisés.

Noms de modules / scripts

1 à 4 segments séparés par ".", caractères [A-Za-z0-9_-], 64 caractères max, suffixe .lua toléré.

`registerMessage(nom, fn [, sortants=false])` → `bool`

Appelle `fn(paquet)` à chaque paquet `nom` (nom clair Dofus 3, ex. "ChatChannelMessageEvent", nom Dofus 2 ou identifiant public Dofus 2 — voir l'encadré). Le global garde UN gestionnaire par message (il remplace ceux déjà posés pour ce message, y compris par `developer:registerMessage`) et n'écoute que les paquets entrants, sauf `sortants = true`. Avec le global, un champ absent vaut `nil`. `developer:registerMessage` AJOUTE un gestionnaire (plusieurs par message, comme SnowBot : l'appeler dans une boucle le fait appeler N fois), voit les deux sens par défaut (sauf `sortants = false`) et reçoit des tables avec défauts `proto3` (voir l'encadré). 64 gestionnaires au total (chaque appel de `developer:registerMessage` compte). Message sensible : `false`.

```
registerMessage("ChatChannelMessageEvent", function(p) print(p.conten
```

Copier

`unRegisterMessage(nom)` → `bool`

`unregisterMessage(nom)` → `bool`

Retire TOUS les gestionnaires du message `nom` (ceux du global comme ceux de `developer:registerMessage`).

`waitFor(nom [, nomEchec] [, timeoutMs=10000] [, action])` → `paquet` | `false`, `paquetEchec` | `nil`

Bloquant

Attend un paquet (nom Dofus 3 ou Dofus 2) ; `action` (fonction) est appelée après l'armement, donc aucune réponse n'est manquée. `nil` = délai écoulé (borné 50 ms – 120 s) ; renvoie `nil` si appelé depuis un gestionnaire ou pour un message sensible. Champ absent du paquet = `nil`.

```
local p = waitFor("ChatChannelMessageEvent", 30000)
if p then print("reçu : " .. (p.content or "")) end
```

Copier

waitForAny({noms} [, timeoutMs=10000] [, action]) → nom, paquet | nil

Bloquant

Attend le PREMIER de plusieurs paquets : renvoie le nom demandé qui est arrivé et sa table, nil au délai (50 ms – 120 s). La table rendue a les défauts proto3 (consommateur façon SnowBot). action : comme waitFor. Nom inconnu : signalé dans la console (« message ... inconnu du protocole », comme waitFor / suspendScriptUntil) et écarté ; nom sensible : refusé (voir « Messages sensibles refusés ») ; aucun nom valide : nil sans attendre. Aussi developper:waitForAny.

```
local nom, p = waitForAny({ "FightEndEvent", "ExchangeLeaveMessage" }, 10000)
if nom then print("reçu : " .. nom) end
```

Copier

lastMessage(nom) → table|nil

Dernière instance REÇUE de ce message depuis la connexion (nil sinon), sans attendre, en table avec défauts proto3 (façon SnowBot). Ex. relire la question d'un PNJ après talkNpc. Message sensible (jeton, ticket...) : jamais mémorisé, nil. Aussi developper:lastMessage.

```
local q = lastMessage("NpcDialogQuestionMessage")
if q then print("message " .. q.messageId) end
```

Copier

require(nom) → module

Charge %APPDATA%/MizanBot/scripts/lib/<nom>.lua ("a.b" = sous-dossier) dans le même bac à sable ; 512 Ko max, liens refusés, cache par run, cycles détectés.

```
local utils = require("utils") -- scripts/lib/utils.lua
```

Copier

dofile(nom) → valeurs

load(texte [, nomChunk]) → fonction | nil, erreur

loadstring(texte) → fonction | nil, erreur

Compat SnowBot, dans le même bac à sable. dofile exécute le fichier de même NOM (un chemin SnowBot est réduit au nom, .lua toléré) cherché dans scripts/lib puis scripts (512 Ko max, pas de lien) et rend ses valeurs ; erreur Lua si introuvable. Pas de cache : ré-exécuté à chaque appel (≠ require). Un fichier déjà en cours d'exécution (lui-même ou un cycle) ou plus de 16 niveaux imbriqués : erreur Lua « récursion » rattrapable par pcall (au lieu de faire tomber le processus). load / loadstring compilent un TEXTE et rendent la fonction, ou nil + message d'erreur (fichier ou fonction-lecteur refusés).

```
dofile(global:getCurrentScriptDirectory() .. "\\config.lua") -- = Copier /
local f = load("return 1 + 1")
print(f())
```

loadScript(nom) → bool
loadAndStart(nom) → bool
reloadScript() → bool

Remplace le script principal : bibliothèque du planning du perso, sinon %APPDATA%/MizanBot/scripts/<nom>.lua. loadAndStart essaie d'abord l'argument TEL QUEL (un nom de la bibliothèque du planning peut contenir « / » ou « \ »), puis le réduit à son nom de fichier (chemin SnowBot) ; un chemin disque manifeste (C:\..., \..., ./, ../, scripts\... de getCurrentScriptDirectory()) est réduit d'emblée. reloadScript() relance le script principal.

```
if getCharacterLevel() >= 50 then loadScript("farm_50") end
```

```
function messagesRegistering()
    registerMessage("ChatChannelMessageEvent", function(p)
        print((p.sender_name or "?") .. " : " .. (p.content or ""))
    end)
end
```

Module developer: — paquets (compat SnowBot)

Attentes et historique de paquets façon SnowBot, en plus de registerMessage / unregisterMessage / waitFor / waitForAny / lastMessage (catégorie Plateforme, aussi sous developer:). Noms de messages Dofus 3, Dofus 2 ou identifiant public Dofus 2 ; messages sensibles refusés (voir Plateforme). Les requêtes HTTP (getRequest...) sont dans le module http:.

Refusés par conception

sendMessage et cancelMessageToClient renvoient false : un paquet forgé librement contournerait les droits vérifiés au point d'action et les gardes du moteur.

unCancelMessageToClient est sans effet (true) ; dofusRequest (API Ankama) renvoie "".

createMessage(nom) ne rend qu'une table { __name } (rien ne peut l'envoyer) ; nil pour un message sensible.

```

developer:suspendScriptUntil(nom | id, délaiMs [, afficherDélai=true]
[, bypass] [, écoute=true]) → bool
developer:suspendScriptUntil(nom | id, délaiMs, afficherDélai,
texteDélai [, pas] [, écoute=true]) → bool

```

Bloquant

Attente **BOULÉENNE** SnowBot, deux formes comme chez SnowBot : si le 4e argument est une chaîne, c'est **texteDélai** (le **failMessageName** de SnowBot : ligne affichée au délai à la place de l'avertissement par défaut, **JAMAIS** un message d'échec) et écoute est le 6e argument ; sinon (**bypass**) écoute est le 5e. **bypass** et **pas** sont ignorés. **true** seulement si le message **nom** arrive avant le délai (défaut 10 s, borné 50 ms – 30 min), **false** sinon. **Nom** inconnu ou refusé : avertissement (une fois), puis attente de **TOUT** le délai et **false** (parité SnowBot). **afficherDélai** = **false** : rien n'est affiché au délai. écoute = **true** (défaut) : pose d'abord la marque (comme **listenPackets()**) et attend un paquet **NOUVEAU** ; écoute = **false** : tout paquet reçu depuis la dernière marque compte, même arrivé avant l'appel. Le paquet reconnu est **CONSOMMÉ** (voir **listenPackets**). **BasicNoOperationMessage** (id 176) = pause d'environ 200 ms puis **true**. Au-delà de 120 s, l'attente est signalée à l'anti-blocage (voulue, pas un blocage).

```
if developer:suspendScriptUntil("GameFightEndMessage", 600000) then printSc
developer:suspendScriptUntil("NpcDialogQuestionMessage", 5000, true, "1e PNJ
```

```

developper:suspendScriptUntilMultiplePackets({noms}, délaiMs [,
afficherDélai] [, bypass | pas] [, écoute=true]) → string

```

Bloquant

Attends le premier des messages listés (4e argument ignoré ; noms inconnus ou refusés signalés une fois et écartés) : renvoie le NOM demandé qui est arrivé, ou la CHAÎNE "False" au délai — aussi quand aucun nom n'est reconnu, après avoir attendu tout le délai (sémantique SnowBot : teste `~ = "False"`, pas not). écoute : comme `suspendScriptUntil` (paquet reconnu consommé).

```
local r = developer:suspendScriptUntilMultiplePackets({ "GameFightEndMessage"
```

```
if r ~= "False" then print("reçu : " .. r) end
```

developer:listenPackets()

Pose la marque d'écoute (SnowBot ListenPackets). Les paquets entrants sont enregistrés EN CONTINU (le dernier de chaque message) ; seuls ceux reçus APRÈS la marque comptent pour un suspendScriptUntil / suspendScriptUntilMultiplePackets appelé avec écoute = false (envoyer, puis attendre sans rater une réponse rapide). Chaque attente avec écoute = true (défaut) repose aussi la marque. Un paquet reconnu par une attente est CONSOMMÉ : une 2e attente écoute = false du même message bloque jusqu'à un NOUVEAU paquet.

developer:historicalMessage(nom | id) → table[]

developer:historicalMessageDofus3(nom) → table[]

Paquets de ce type reçus pendant la CONNEXION au serveur de jeu, jusqu'au 1er chargement de carte (caractéristiques, inventaire, métiers, quêtes...), du plus ancien au plus récent, en tables avec défauts proto3 ; {} si aucun ou si le message est sensible (2 000 paquets / 4 Mio retenus au plus). Changement de perso en jeu : l'instantané repart pour le NOUVEAU perso.

developer:typeoff(valeur) → string

developer:toggleMultiThreading()

developer:isScriptPlaying() → bool

developer:hasScript() → bool

developer:startScript() → bool

typeoff : __name d'une table de paquet, sinon le type Lua ("number", "table"...).

toggleMultiThreading : sans effet (les hooks sont servis aux points sûrs du thread du script).

hasScript → true ; startScript relance le script principal s'il ne tourne pas (true s'il tourne).

developer:getNameByIdentifier(idD2) → string

developer:getIdentifierByName(nomD2) → string

developer:createMessage(nom | id) → table|nil

Identifiant PUBLIC Dofus 2 → nom Dofus 2, et l'inverse (chaîne, "" si inconnu) — jamais un code du protocole courant. createMessage : { __name = nom Dofus 3 } (lecture / composition seulement), nil pour un message sensible.

Modules http: & developer: — requêtes web

Permission « Réponse automatique ». https uniquement, adresses publiques uniquement (réseau local refusé), pas de redirection suivie ni de cookie, 10 s max, 256 Kio envoyés / 1 Mio reçus, connexion directe (pas le proxy du compte).

`http:get(url [, headers]) → table`

`http:post(url, body [, headers]) → table`

Bloquant

Perm.PmRelay

Renvoie { ok, status, body, contentType, location, error } ; headers = { ["Nom"] = "valeur" }.
error ∈ no_permission, bad_url, blocked_address, bad_header, too_large, timeout, network.

```
local r = http:get("https://api.example.com/prix?gid=289")
if r.ok then local t = json.parse(r.body) end
```

Copier

`developer:getRequest(url [, noms, valeurs]) → string`

`developer:postRequest(url, data [, noms, valeurs]) → string`

`developer:getRequest2(url [, noms, valeurs]) → string`

`developer:getRequestWithDelay(url [, délaiMs]) → string`

Bloquant

Perm.PmRelay

Compat SnowBot : renvoie le corps (" " en échec) ; les arguments proxy SnowBot (5e, 6e) sont ignorés (connexion directe). getRequest2 = getRequest (SnowBot passait par le proxy du compte). getRequestWithDelay renvoie le corps ou "Error: <raison>" (délai ignoré). Aussi snowbotController:getRequest / postRequest. Les verbes de paquets de developer: sont dans « Module developer: ».

Trajet déclaratif (move() / bank())

move() (et bank()) peut renvoyer une table d'étapes. À chaque tick, le moteur : 1) trouve l'étape de la carte courante — d'abord par id EXACT, puis par coordonnée "x,y" (tirets unicode et espaces tolérés) ; plusieurs étapes pour la même carte sont FUSIONNÉES clé par clé (la dernière qui définit une clé gagne) ; 2) joue ses clés dans l'ordre ci-dessous ; 3) suit changeMap / path vers la carte suivante.

Étapes en double (SnowBot)

Plusieurs entrées pour la même carte forment UNE étape, fusionnée clé par clé : la DERNIÈRE

qui définit une clé gagne (comme SnowBot). Ex. { map = "-15,39", gather = true } puis { map = "-15,39", path = "right" } donnent gather ET path ; deux path différents → celui écrit en dernier ; door dans l'une et path dans l'autre → les deux (door joué d'abord). Jamais de mélange id / coordonnée : dès qu'une étape vise l'id exact, les étapes à la même coordonnée sont ignorées (un intérieur partage la coordonnée de l'extérieur).

Valeurs de path / changeMap

Direction "top" / "bottom" / "left" / "right" (synonymes FR/EN acceptés : haut, bas, gauche, droite, nord, sud, est, ouest...) ; direction + cellule "top(12)" ; alternatives "left|right" (une au hasard) ; liste { "left", "right" } (au hasard, en évitant de revenir sur la carte précédente) ; liste pondérée { { direction = "left", weight = 50 }, { path = "right", weight = 30 } } ; coordonnée "4,-18" (adjacente = direction, lointaine = voyage) ; id de carte 88085761 ou "88085761" (voyage) ; liste d'ids { 154133, 154134 } (un au hasard) ; téléports "zaap(<mapId>)", "zaapi(<mapId>)", "havenbag". Préfixes tolérés : goto:, travel:, map:, zaap:, usezaap:, changemap:.

Téléports et contournements

zaap(id) : zaap présent sur la carte → téléport direct ; sinon voyage vers la carte. Une coordonnée adjacente sans sortie réelle dans le world-graph (falaise) est traitée comme un voyage (contournement). Sur la carte havre-sac sans étape dédiée, le bot ressort automatiquement.

Hors trajet & blocages

Aucune étape pour la carte courante : si une étape porte un id numérique, le bot y voyage (zaap si utile) ; sinon il rejoint la carte du trajet la plus proche joignable. Après 3 échecs de voyage vers la même carte, ou 5 échecs hors trajet consécutifs, le script s'arrête (raison stuck) et le planning se met en pause. Carte hors world-graph (instance, intérieur) : attente, puis arrêt après ~15 itérations.

Coffres et maisons à code

Séparateur | ou ; pour lockedStorage / lockedHouse ; code "-1" ou "" = pas de code. Mauvais code, propriétaire introuvable ou format invalide = arrêt du script ; coffre occupé = nouvel essai 5 s plus tard ; 5 échecs = arrêt.

CHAMP	TYPE	RÔLE
map	string number	Carte de l'étape (obligatoire) : coordonnée "x,y", id (nombre ou chaîne) ou "havenbag" (le havre-sac).

<code>regeneration</code>	<code>number</code> <code>bool</code>	Régénère si PV% < n (true = MIN_LIFE_PERCENT, sinon 100), jusqu'à REGEN_UP_TO_PERCENT.
<code>gather</code>	<code>bool</code>	Récolte une ressource par itération tant qu'il en reste (filtre ELEMENTS_TO_GATHER), puis suit path.
<code>forcegather</code> / <code>forceGather</code>	<code>bool</code>	Comme gather, puis RESTE sur la carte à attendre les repousses (path / door / custom ignorés ; sortie par pods pleins, combat ou arrêt).
<code>fight</code>	<code>bool</code>	Attaque le groupe éligible le plus proche (filtres MIN/MAX/FORBIDDEN/FORCE_MONSTERS) avant de suivre path.
<code>forcefight</code> / <code>forceFight</code>	<code>bool</code>	Comme fight, puis reste sur la carte à attendre un groupe éligible (groupes injoignables écartés).
<code>npcBank</code>	<code>bool</code>	Le perso est sur la carte du banquier PNJ → dépôt de tout le sac (séquence banquier).
<code>lockedStorage</code>	<code>string</code>	"cell code[skillId]" : coffre à code ; ouvre, dépose (mode de l'onglet Banque), ferme, puis poursuit l'étape.
<code>lockedHouse</code>	<code>string</code>	"cell code[owner]" ou "owner code" : entre dans une maison à code (le tick suivant joue l'étape de l'intérieur).
<code>cell</code>	<code>number</code>	Se positionne sur cette cellule avant la suite de l'étape.
<code>custom</code>	<code>function</code>	Bloc Lua libre. S'il change de carte / lance un combat, l'étape s'arrête là ; s'il RENVOIE une table d'étapes, ce sous-trajet est joué (profondeur max 3).
<code>lockedCustom</code>	<code>function</code>	Comme custom, sous VERROU D'ÉQUIPE : un seul membre à la fois (attente max 120 s).
<code>door</code>	<code>number</code> <code>string</code>	Utilise l'interactif de cette cellule (porte, escalier...) pour changer de carte ; "c1 c2" = une au hasard.
<code>exitCell</code>	<code>number</code>	Quitte la carte par cette cellule de bord (= path "dir(cell)").
<code>changeMap</code> / <code>path</code>	<code>string</code> <code>number</code> <code>table</code>	Sortie vers la suite (voir « Valeurs de path »). changeMap prime sur path.
<code>paths</code>	<code>string</code> <code>number</code> <code>table</code>	Nom SnowBot (chef + équipe) : lu comme path quand changeMap et path sont absents ; le suivi d'équipe fait suivre les membres.

phenix

number |
string

Seulement dans le trajet renvoyé par phenix() : cellule de la statue du Phénix à utiliser, ou "c1|c2" (une au hasard) ; prioritaire : les autres clés de l'étape sont alors ignorées — voir Cycle de vie & callbacks.

```
ELEMENTS_TO_GATHER = { 258, 261 }  -- Mandragore, Millet  
MAX_PODS = 90
```

Copier

```
function move()  
  return {  
    { map = "212861955", gather = true, path = "171443204" },      -- Zaa  
    { map = "-15,39", gather = true, fight = true, path = { "right", "top" } },  
    { map = "-15,38", regeneration = 50, forcegather = true },      -- can  
    { map = "-14,39", gather = true, path = "left|bottom" },  
    { map = "-14,40", custom = function()  
      local pnj = map:getNpcs()[1]  
      if pnj then talkNpc(pnj.id) replyNpc(1) leaveDialog() end  
    end, door = 312 },      -- pu  
  }  
end  
  
function bank()  
  return {  
    { map = tostring(BANQUE_MAP_ID), npcBank = true },      -- id exact : le bot  
  }  
end
```

Compatibilité SnowBot

Un script SnowBot est conçu pour tourner SANS modification : toutes ses tables existent (global:, config:, snowbotController:, map:, npc:, chat:, inventory:, exchange:, sale:, craft:, merchant:, character:, mount:, quest:, treasure:, d2data:, developer:, fightAction:, fightCharacter:, fightBasic:, fightDebug:, fightSlave:, fightChallenge:), en appel colon ou point, avec des noms tolérants (sale:sellItem = SellItem) ; les noms de messages Dofus 2 sont acceptés partout ; les callbacks move, bank, phenix, fightManagement, prefightManagement, challengeManagement, messagesRegistering et stopped sont appelés. Les alias historiques restent (goToMapId, useInventoryItem, deleteItem, map:getPathDistance, inventory:equiItem, quest:questActive, character:Upgrade*, combat:surrender...). Différences de sémantique à connaître :

HDV (sale:)

GetItem* 1-based (for i = 1, sale:ItemsOnSale()), 0 accepté comme alias de 1. GetPriceltem / getAveragePriceltem : lot = index 1..4 ou taille ; getAveragePriceltem multiplie le prix moyen

unitaire par la taille du lot ; GetLots renvoie un index 1..4. UpdateAllItems REPOSITIONNE les prix (sous-cote de 1, jamais à la hausse, jamais contre toi-même) ; l'ancienne lecture seule = refreshPrices(). SellItem prend une quantité, pas un index. Sentinelles : prix inconnu = -1 (SnowBot : 0), getGID d'un GUID inconnu = 0 (SnowBot : -1), buyItem avec prixMax = 0 = sans plafond. Ouvrir le HDV : goToHdv / goToHdvBuy (interactif de l'hôtelier puis bascule), ou hôtelier (interactif) ouvert par le script puis npc:npc(-1, 5 / 6), qui envoie la même requête de bascule vente / achat que goToHdv (permission « Hôtel de Vente »). AvailableSpace est réel (-1 tant qu'aucun HDV vendeur n'a été ouvert).

PNJ

SnowBot raisonne en ids de MODÈLE de PNJ : talkNpc et npc:* acceptent le modèle (> 0) comme l'id d'acteur (négatif, map:getNpcs()), résolu sur la carte ; 0 = premier vrai PNJ (jamais un indice de chasse au trésor) ; un modèle absent de la carte = introuvable (false tout de suite, npcBank compris). npc:reply(n) : n < 0 = index, n > 0 = identifiant de réponse ; replyNpc(i) = toujours un INDEX 1-based. talkNpc attend la question, replyNpc la suivante (≤ 2 s).

Messagerie, banque & échanges

sendPrivateMessage global : ordre MizanBot (pseudo, message), l'ordre SnowBot est corrigé si le message contient un espace et le pseudo aucun (un message d'un seul mot ne peut pas être détecté) ; chat:sendPrivateMessage(contenu, destinataire) garde l'ordre SnowBot sans correction. exchange:getKamas(n) RETIRE du coffre ; bank:getKamas() lit le solde. exchange:putItem va aussi vers l'établi de craft ouvert (une erreur d'échange n'y compte pas comme une pose) ; putItem / putAllItems / getAllItems attendent l'accusé adapté comme SnowBot (échange ≤ 1,5 s ; coffre : sac réellement diminué, ou rafale de mises à jour retombée) ; un retrait d'échange ne reprend que ce qui a été posé. inventory:getAveragePrice est propre au serveur de jeu du bot.

Mémoire, compte & identifiants

global:addInMemory... = RAM TYPÉE par (serveur de jeu, PERSONNAGE) — deux homonymes sur deux serveurs ne la partagent pas —, conservée à travers les rechargements, reconnect*, PLANNING et relance ; *GlobalMemory (tous les bots) et snowbotController:addInMemory (toute l'app) aussi ; tout est perdu à la fermeture de l'app. Une table rangée est la MÊME table pour le run qui l'a rangée (mutations en place conservées, comme SnowBot), une copie pour les autres scripts. memory: / sharedMemory: = disque, en CHÂÎNES. username() / getAlias() et les listes « Username » de snowbotController: rendent l'ALIAS du compte, jamais l'identifiant ; password(), getToken(), getPassword, les verbes du code Bouclier et snowbotController:registerAnAccount / createCharacter / assignProxyToAnAccount / unlockShield / addPushBulletKey sont refusés ("" / false, « refusé par sécurité » dans la console).

Carte, quêtes, perso, inventaire

map:countPlayers() inclut ton perso (#map:playersOnMap() non ; map:players() l'inclut, forme SnowBot). map:currentAreald() renvoie le VRAI id de zone (SnowBot : l'id de sous-zone), currentArea() le NOM de la zone (SnowBot : l'id de zone en chaîne), currentSubAreald() sans argument l'id de sous-zone (SnowBot : l'id de zone) ; currentSubArea() = nom de la sous-zone des deux côtés. map:Forbiddlle suit SnowBot (zones différentes refusées seulement si l'une est une île ; zone inconnue → false). character:breedName() rend le nom accentué (« Féca », « Xélor », « Crâ ») : compare breed(). Objectifs de quête : reached = true = fait, nil = inconnu. character:sex() : true = femme. inventory:useItem(gid) utilise 1 exemplaire (SnowBot) ; inventory:use(gid) et le global useItem(gid), tout le stack. inventory:podSP() = 100 si le poids max est inconnu. goto est un mot-clé Lua : zaap:go / zaapi:go.

Boucle, banque & réglages

move() est appelée en boucle (~200 ms et après chaque ressource), pas une fois par arrivée sur la carte : un print ou un compteur en tête de move() se répète. Carte sans étape : SnowBot arrête le script, MizanBot rejoint la carte du trajet la plus proche. SnowBot n'a pas de MAX_PODS (seuil de banque = réglage d'interface, 90 % par défaut, déclenché pour toute l'équipe) : sans MAX_PODS, MizanBot banque à 95 %, par bot. MAX_MONSTERS absent = 8 chez SnowBot, 999 ici. OPEN_BAGS : SnowBot ouvre les sacs à chaque tour, MizanBot seulement pods pleins ; AUTO_DELETE ne vaut que pour le bot du script (SnowBot le poussait à l'équipe). config:set* reste posé (SnowBot le réécrivait avant chaque move()).

Options opt-in & planning

FIGHT_KICK (expulsion au placement, aussi pendant fight() et les combats de trajet) et IGNORE_REQUEST_GUILD (refus des invitations de guildes, sauf celles de tes propres persos) sont OPT-IN chez MizanBot, alors que SnowBot les fait par défaut : déclare-les pour retrouver ce comportement. PLANNING liste des heures de DÉCONNEXION (sémantique SnowBot), indépendamment du planning de l'onglet ; une pause demandée par PLANNING ou reconnect* n'est pas défaite par la connexion auto du planning, mais un Arrêter / Lancer manuel l'annule. SNOWBOT_TIMEOUT est en TOURS SnowBot de 6 s (valeur × 6 s, 60 s minimum) et remplace le seuil anti-blocage du run (sans effet si l'anti-blocage est désactivé dans les réglages du bot).

Combat

fightManagement = onTurn ; prefightManagement / challengeManagement (si onPlacement est absent) sont des NOTIFICATIONS : le moteur fait ensuite son pré-combat si « pré-combat auto » est actif dans le profil (placement selon le style sauf si le script s'est placé, défi, prêt), comme SnowBot ; arguments de prefightManagement au format SnowBot { [cellule] = occupant | -1 }. fightBasic:addSpell (TeamEnum SnowBot 0-15) / playTurn font jouer l'IA

MizanBot avec la rotation du script (pas l'IA SnowBot) ; fightAction: rend ses listes en tableaux de nombres, getCells_* ne rend que des cases marchables, isWalkable ignore l'occupation, getFighter rend -1 sur une case vide, getAllEntities().Team = true pour un ENNEMI (sens SnowBot) ; fightSlave: est neutre. Un fightManagement() qui plante rend la main au moteur pour le reste du run.

Paquets (developer:)

Noms Dofus 2 (« ...Message ») et identifiants publics Dofus 2 acceptés partout (alias limités aux messages du catalogue Dofus 3 courant). Tables de paquets = tables Lua : listes 1-based, clés snake_case d'origine, camelCase / PascalCase tolérés, t.Count = nombre d'éléments d'une liste ; défauts proto3 (0 / false / "" / {}) pour les champs absents du message racine seulement chez les consommateurs façon SnowBot (developer:registerMessage, waitForAny, lastMessage, historicalMessage, character:getStats) — registerMessage / waitFor globaux gardent « champ absent = nil ». Messages porteurs de secrets (jetons, tickets, identification...) refusés partout. developer:registerMessage AJOUTE un gestionnaire et voit les deux sens ; le registerMessage global en garde un seul (entrants). suspendScriptUntil rend un booléen (4e argument texte = ligne affichée au délai, jamais un message d'échec ; nom inconnu = attente de tout le délai), suspendScriptUntilMultiplePackets un nom ou "False" ; paquets enregistrés en continu, marque d'écoute et paquet consommé comme chez SnowBot. Les hooks observent seulement (pas d'interception / réémission) ; messagesRegistering() est toujours appelée (SnowBot : seulement si son nom figurait dans le texte).

Trajet & fichiers

DSL de trajet SnowBot supporté : "a|b", "dir(cell)", "zaap(id)", "zaapi(id)", "havenbag", door, exitCell, forcegather, forcefight, regeneration, lockedStorage, lockedHouse, lockedCustom, npcBank, paths, phenix (dans phenix()), custom qui renvoie un sous-trajet, et étapes en double fusionnées (la dernière définition d'une clé gagne). map:moveToward / moveNext font le trajet COMPLET (SnowBot : une transition par appel). Les arguments proxy HTTP de SnowBot sont ignorés ; dofile / loadAndStart résolvent un NOM de fichier (loadAndStart essaie d'abord l'argument tel quel), jamais un chemin disque ; dofile refuse la récursion.

Non supportés

mule_lost (jamais appelé : suivi d'équipe MizanBot), SYNC_MOVE / AI_FILE (sans équivalent : suivi d'équipe et IA de l'onglet Combat), map:aggression (agression PvP de masse, non proposée), developer:sendMessage / cancelMessageToClient (sécurité), Kolizéum et questions modales askQuestion / askInput / rcAsk* (sans équivalent : valeurs neutres), boutique PNJ sale:*ToPnj et montures mount: (messages de jeu à capturer), treasure.nextMapId (résolveur d'indices tiers non porté). Liste complète : « Non supporté ».

Non supporté

Ces verbes existent (pour éviter « attempt to call a nil value ») mais n'agissent pas : la plupart journalisent un avertissement (une fois) et tous renvoient false / nil / une valeur neutre. Trois raisons : refus de sécurité, message de jeu à capturer (aucun paquet deviné), ou aucun équivalent MizanBot.

Non implémentés (absents de l'API)

askConfirm, lecture du contenu d'un coffre à code depuis Lua, envoi de paquets bruts (developer:sendMessage est refusé).

REFUSÉS — SÉCURITÉ (IDENTIFIANTS, COMPTE, PAQUETS LIBRES, DISQUE)

global:password() → "" global:getToken() → "" snowbotController:getPassword → ""

character:askShieldCode / sendShieldCode / askShieldCodeNew / sendShieldCodeNew → false

snowbotController:registerAnAccount / createCharacter / assignProxyToAnAccount / unlockShield / addPushBulletKey → false (« refusé par sécurité » dans la console)

developer:sendMessage / cancelMessageToClient → false (unCancelMessageToClient → true, sans effet)

developer:dofusRequest → ""

d2data:exportD20 / exportAllD20 / exportMapData / exportAllMapData → false

messages porteurs de secrets (jetons, tickets, clés d'API, identification...) → refusés : registerMessage / unregisterMessage → false, waitFor / lastMessage / createMessage → nil, historicalMessage → {}, suspendScriptUntil* → false / "False" au délai

executeLUA sur son propre compte → false (et sur un compte hors de ton équipe en jeu : false + avertissement)

global:getAllFileNameInDirectory avec une extension autre que 1 à 10 lettres / chiffres → {}

MESSAGE DE JEU À CAPTURER

sale:buyToPnj / sellToPnj → false ; sale:sellAllItemToPnj / sellAllItemToPnjSecret → nil (boutique PNJ)

mount:* (montures, enclos, élevage) → false / -1 / nil / 0 / {} – voir le module mount:

combat:setChallengeBonus → false (le bonus suit le choix mémorisé du client)

SANS ÉQUIVALENT MIZANBOT (VALEUR NEUTRE)

```

map:aggression → false (agression PvP de masse non proposée)

global:isKoliServer / isStuffForKoli / isRegisteredInKoli /
unregisterFromKoliIfRegistered → false (Kolizéum)

global:askQuestion / rcAskFight / rcAskGather → false ; global:askInput → ""
(questionnaires modales)

global:checkModerator / antiModeratorActivated → false ; activateAntiModerator → nil
(voilà MODERATOR_STOP / MODERATOR_WEBHOOK)

snowbotController:activateAntiModerator / loadTeamAccount / loadTeamAccountbak /
loadControllerAccount → false

global:editAlias / loadConfiguration / LoadConfigurationWithoutScript /
groupCharacters / ungroupCharacters / openMitm → false

global:blabla → nil ; global:accountTag → ""

fightSlave:* → -1 / "" / nil / {} / false (invocations contrôlées non pilotées)

merchant:open → nil ; buyItem / existItem → false ; priceItem / quantityItem → -1
(inactif chez SnowBot aussi)

character:getCost* / getSpellMinPlayerLevel / accountId → -1 ; nickname /
getURLFaceSkin / getURLBodySkin → "" ; trusted / certified → false

d2data:objectFromD20 → nil ; allObjectsFromD20 → {} (pas d'accès générique aux D20)

quest:objectiveText → "" ; quest:path → nil ; quest:mainPath → ""

treasure.nextMapId → 0 et résolveur d'indices tiers de SnowBot non porté
(treasure:gdPggXxViU → nil) ; lastHuntReport → {} ; sendLastHuntReport /
printLastHuntReport / treasureInactivity → sans effet

fightAction:forceCellidEntity, fightBasic:setFightSpeed,
developer:toggleMultiThreading → sans effet

map:checkMonsterAmountConstraints → true (MONSTERS_AMOUNT appliqué par le moteur) ;
map.Precompute → false ; sale.itemToUnfollow → 0

```

JAMAIS APPELÉS / IGNORÉS

```

callback mule_lost(carteDuChef) (suivi d'équipe MizanBot)

globals SYNC_MOVE et AI_FILE script « contrôleur » SnowBot sans compte (run())

interception de paquets sortants (les hooks observent seulement)

```